# AgentGate

An explainable, bounded, and gated AI agent commerce layer — built for the Razorpay AI Buildathon, Track 1: AI Growth & Agentic Commerce

---

## 1. Purpose & Problem Statement

The track brief asks for an agent that makes a merchant transactable by an AI buyer end to end, on Razorpay's test-mode APIs — with every money action explainable, bounded, and gated, and at least one failure handled gracefully. Most naive implementations treat this as a thin wrapper: an LLM calls a checkout API directly. AgentGate treats the space between agent intent and Razorpay execution as a first-class problem — the same way a network engineer treats the space between a packet's origin and its destination.

AgentGate is a trust-and-inspection layer that sits between an AI shopping agent and Razorpay's test-mode payment APIs. Every transaction request from the agent is inspected, scored against spend bounds and behavioral rules, logged with full reasoning, and only then allowed to proceed to Razorpay. Requests that fail inspection are blocked with a clear, logged explanation, and the agent is expected to recover gracefully rather than fail silently.

## 2. Track Alignment

| Track requirement | How AgentGate addresses it |
|---|---|
| Agent makes merchant transactable end to end | Agent reads a catalog, converses, and completes a real test-mode Razorpay order |
| Every money action explainable | Each gate decision (allow/block) is logged with the reasoning behind it |
| Every money action bounded | Per-session spend limits, quantity limits, and rate limits enforced before Razorpay is called |
| Every money action gated | No agent tool call reaches Razorpay directly — all requests pass through the gate fi |
| Show audit trail | SQLite-backed log of every decision, timestamp, and Razorpay response |
| One failure handled gracefully | A deliberately triggered bound violation (e.g. over-limit order) is caught, explained, and rec |

## 3. Why This Design (Prior Work Applied)

AgentGate's architecture is a direct extension of two systems already built independently of this hackathon, applied to a new domain rather than invented from scratch:

- **From DPI-Engine** (multithreaded packet inspection engine, pure Python stdlib, 10K+ packets/sec): the practice of inspecting every unit of traffic against rules before it is allowed through, and maintaining per-flow state using consistent hashing on identifying tuples. AgentGate applies the same idea to agent transaction requests instead of network packets — each agent session is tracked, rate-limited, and inspected the way a 5-tuple flow is.

- **From MeshPay** (offline UPI-style payment protocol using public-key cryptography and gossip propagation): the practice of never trusting an action until it passes an explicit verification step. AgentGate's gate plays the same role Meshpay's handshake plays — no transaction proceeds until it is explicitly cleared.

## 4. Functional Requirements

- Expose a small product catalog in a structured, agent-readable format (JSON).
- An LLM-backed agent can converse with a simulated buyer and decide what to purchase.
- The agent acts only through defined tools: list_products, create_order, confirm_payment.
- Every tool call that touches money is intercepted by the gate before reaching Razorpay.
- The gate enforces: per-session spend cap, per-item quantity cap, and a minimum interval between order attempts (rate limiting).
- Approved requests call Razorpay's test-mode Orders API and return a real (test) payment result.
- Blocked requests return a clear, logged reason instead of a silent failure.
- Every decision (allowed or blocked), its reasoning, and the Razorpay response (if any) is written to a persistent audit log.
- At least one failure scenario is deliberately reproducible on demand for the pitch video.

## 5. Tech Stack

### Backend framework

**Python 3.11 + FastAPI.** Chosen for speed of development under time pressure and prior familiarity from TrackitNow. FastAPI's dependency injection is used to route every money-touching call through the gate before it reaches the Razorpay client.

### AI agent layer

**LLM with function/tool calling** (Claude or GPT via API). The agent is given three tools — list_products, create_order, confirm_payment — and reasons over the catalog to decide what to do. The model itself never talks to Razorpay directly; it only ever calls tools, which the gate intercepts.

### Trust / inspection gate

Custom Python module, no external dependency. Performs three checks per request: spend-bound check, quantity-bound check, and rate/session check (modeled on DPI-Engine's per-flow state tracking, using an in-memory dict keyed by session id instead of a 5-tuple).

### Payments integration

**Razorpay Python SDK** (razorpay package) against test-mode API keys. Used for Orders API calls; test-mode credentials mean no real money moves.

### Data storage / audit trail

**SQLite** via Python's built-in sqlite3. Chosen deliberately over Postgres/Supabase: this is a multi-day build, not a production system, and SQLite needs zero setup while still being a real, inspectable, persistent log — every gate decision, timestamp, and Razorpay response is written here.

### Interface

Minimal — a CLI or a single-page HTML view showing the live conversation, gate verdicts, and transaction log. No frontend framework, no styling investment. Reviewers score build quality and AI judgment, not UI polish, so effort is deliberately not spent here.

### Libraries summary

| Purpose | Library |
|---|---|
| Web framework | fastapi, uvicorn |
| Agent / LLM calls | anthropic or openai (official SDK) |
| Payments | razorpay (official Python SDK) |
| Data validation | pydantic |
| Audit storage | sqlite3 (standard library) |
| Environment config | python-dotenv |

## 6. Request Flow

- 1. Buyer (simulated) sends a message to the AI agent.
- 2. Agent reasons over the catalog and decides to call a tool (e.g. create_order).
- 3. The tool call is intercepted by the gate, not sent to Razorpay directly.
- 4. Gate checks: spend bound, quantity bound, rate limit — using per-session state.
- 5. If the gate approves: the Razorpay Orders API is called with test-mode keys, and the result is returned to the agent.
- 6. If the gate blocks: a structured reason is returned to the agent instead, and the agent must explain this to the buyer and attempt a valid recovery (e.g. reduce quantity).
- 7. Every step — approved or blocked — is written to the SQLite audit log with a timestamp and reasoning.

## 7. Deliverables for Submission

- [ ] Public GitHub repo with clean README (what it does, architecture, how to run it).
- [ ] 5-minute pitch video: problem, live demo, architecture diagram, the deliberate failure-recovery moment.

- [ ] Written answer for 'what broke and how you fixed it', based on real build friction, not invented.

---

Track: AI Growth & Agentic Commerce | Project: AgentGate | Razorpay AI Buildathon 2026 | Applications close 5 September 2026